PENETRATION TEST REPORT

Ayman Abdelmonsef

VulnHub: Vulnerable Docker 1

Docker Container Escape & Internal Network Pivoting

Prepared by	Ayman
Date	June 6, 2026
Target	192.168.147.146 (Vulnerable Docker 1)
Platform	VulnHub
Classification	CONFIDENTIAL
Severity	Critical

1. Executive Summary

This report documents the step-by-step exploitation of the VulnHub machine "Vulnerable Docker 1." The assessment demonstrated a full attack chain beginning from open-source reconnaissance on the host network, culminating in root-level access inside a Docker container on an internal (hidden) network segment. The following critical weaknesses were identified and exploited:

- Two open TCP services on the target (SSH on port 22, HTTP on port 8000 running WordPress 4.8.1).
- WordPress user enumeration leading to successful brute-force credential theft (bob / Welcome1).
- Remote code execution via the Metasploit wp_admin_shell_upload exploit, yielding a Meterpreter shell as www-data.
- Discovery of an internal Docker network (172.18.0.0/16) hidden from the host-facing interface.
- Pivoting via Metasploit autoroute + port forwarding to reach 172.18.0.4:8022 — a Docker SSH service running as root with no password.

The entire chain required no CVE-level kernel exploit; misconfiguration and weak credentials were sufficient.

2. Scope & Methodology

The engagement was a black-box assessment of the VulnHub "Vulnerable Docker 1" machine hosted locally in a VMware environment. The tester operated from a Kali Linux attack machine on the same 192.168.147.0/24 subnet.

Phase	Activity
Reconnaissance	Host discovery (fping), port scanning (nmap), service fingerprinting, web directory enumeration (feroxbuster)
Enumeration	WordPress scanning (WPScan), user enumeration, robots.txt review, version fingerprinting
Exploitation	Credential brute-force (WPScan XML-RPC), Meterpreter shell via wp_admin_shell_upload
Post-Exploitation	System survey, SUID enumeration, internal network discovery, LinEnum script upload
Pivoting	Metasploit autoroute, TCP port scan of internal 172.18.0.0/16, port forwarding to Docker SSH service
Privilege Escalation	SSH login to internal container as root (Docker SSH — no password required)

3. Reconnaissance

3.1 Host Discovery & Port Scanning

The attacker began by identifying live hosts on the local subnet using fping, a fast ICMP-based host discovery tool. The target IP 192.168.147.146 was identified as active. An initial Nmap SYN scan (-sS -p-) confirmed two open ports: 22/TCP (SSH) and 8000/TCP (HTTP alternative).

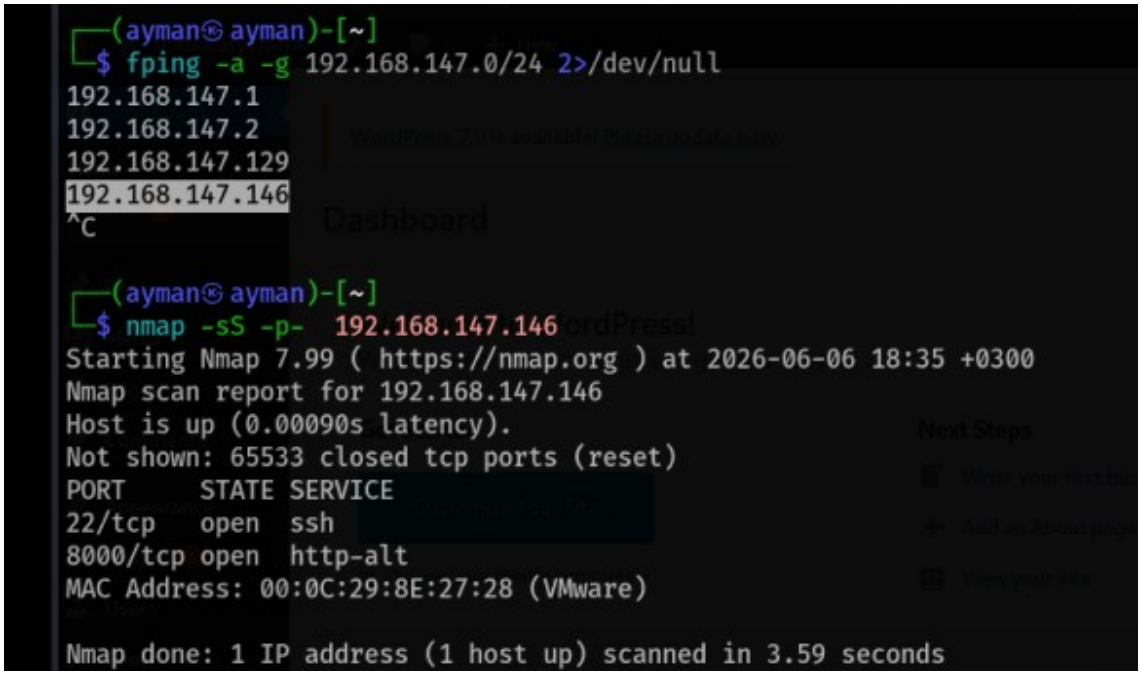

Figure 1 — fping host discovery identifies 192.168.147.146; quick Nmap SYN scan reveals ports 22 (SSH) and 8000 (HTTP).

3.2 Service Version & Banner Fingerprinting

A deeper Nmap service version scan (-sC -sV) was run against 192.168.147.146. This revealed OpenSSH 6.6p1 on port 22 and Apache httpd 2.4.10 on port 8000. Critically, the HTTP service was identified as a WordPress 4.8.1 installation titled "NotSoEasy Docker," with /wp-admin/ listed as a disallowed path in robots.txt.

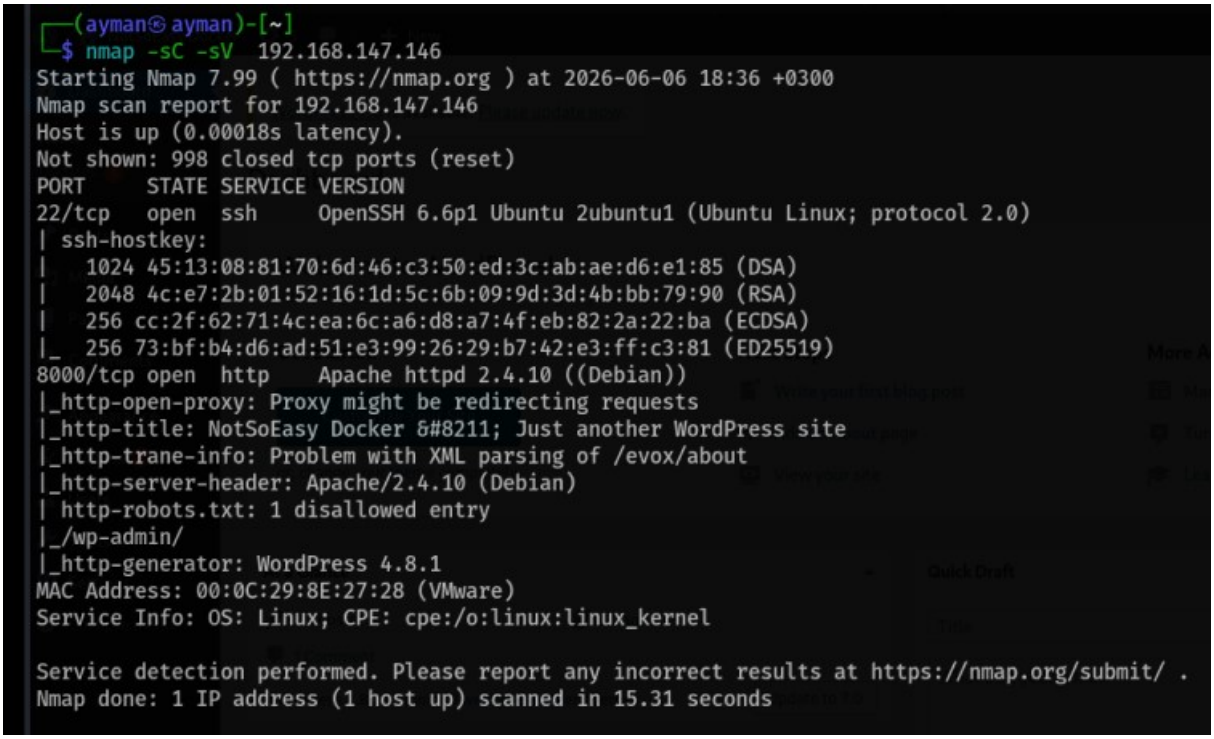

Figure 2 — Nmap -sC -sV reveals WordPress 4.8.1, Apache 2.4.10 (Debian), PHP 5.6.31, and a /wp-admin/ disallow entry in robots.txt.

4. Web Application Enumeration

4.1 Directory & Content Discovery (Feroxbuster)

Feroxbuster was launched against http://192.168.147.146:8000/ using the SecLists quickhits wordlist, filtering HTTP 503 responses. This rapidly mapped accessible WordPress routes including /wp-admin/, /wp-login.php, and several theme/plugin asset paths.

Figure 3 — Feroxbuster command targeting port 8000 with SecLists quickhits wordlist, filtering 503 responses.

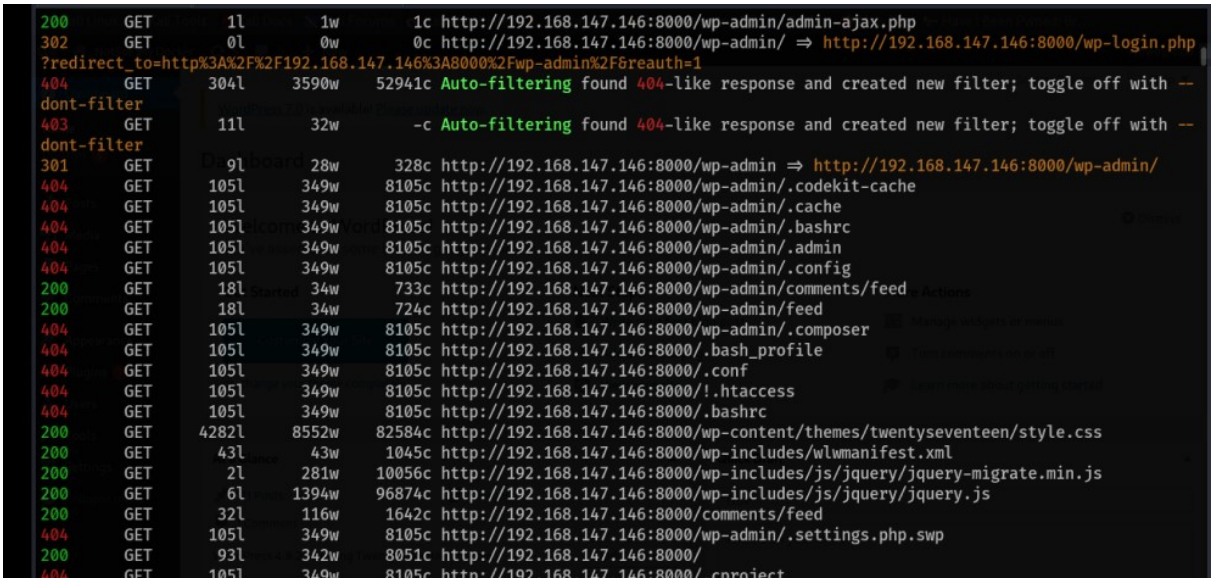

Figure 4 — Feroxbuster results: HTTP 200 responses confirm /wp-admin/admin-ajax.php, feed, comments, and WordPress asset paths. HTTP 302 redirect confirms /wp-admin/ is accessible.

4.2 robots.txt Review

Manual navigation to /robots.txt confirmed the Disallow entry for /wp-admin/ — an indicator that the WordPress admin panel exists and the site owner attempted (ineffectively) to hide it from search engines.

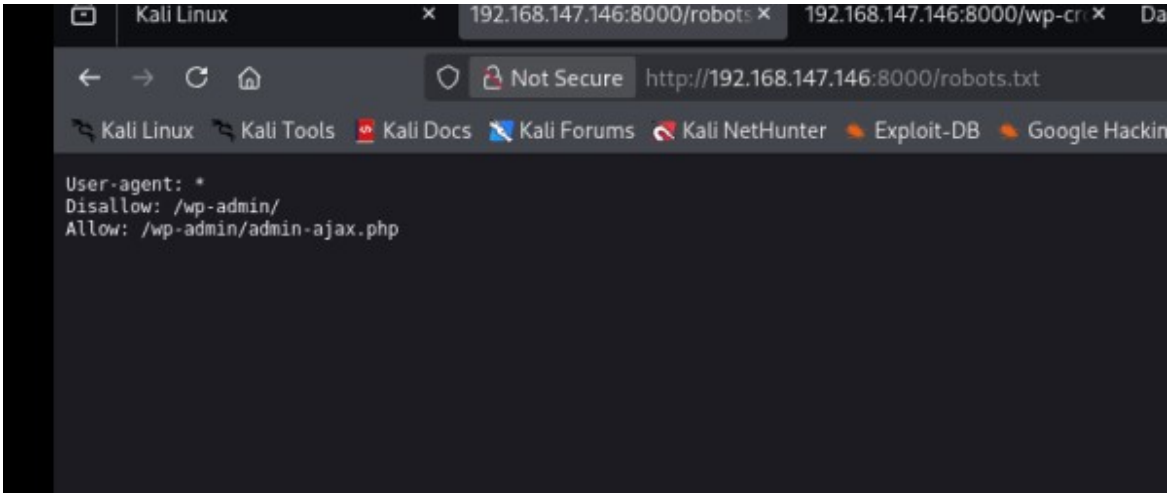

Figure 5 — robots.txt disallows /wp-admin/ while allowing /wp-admin/admin-ajax.php, confirming the WordPress admin panel is present.

4.3 WordPress Scanning (WPScan — User Enumeration)

WPScan was launched in user-enumeration mode (-e u) against the WordPress installation. The scan confirmed the server was running Apache 2.4.10 on Debian with PHP 5.6.31, and identified the WordPress version as 4.8.1.

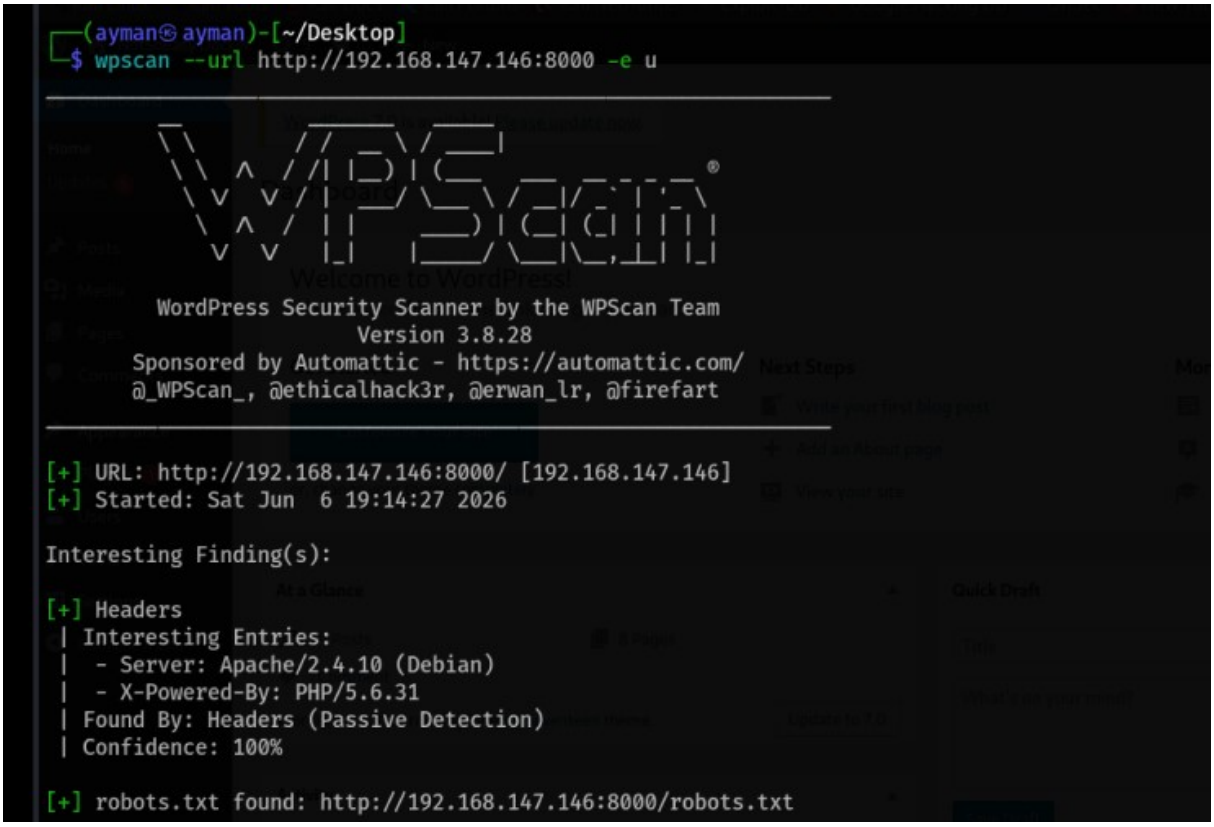

Figure 6 — WPScan startup, target URL confirmed, server header (Apache 2.4.10 / PHP 5.6.31) fingerprinted via passive detection.

WPScan's user enumeration successfully identified the WordPress username "bob" through multiple passive and aggressive detection methods including RSS generator output, the WP JSON API, and author post pattern matching.

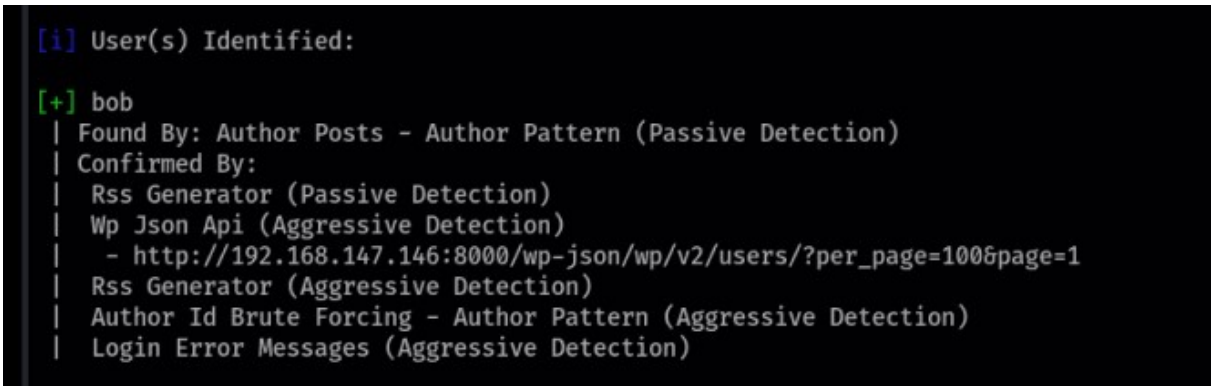

Figure 7 — WPScan identifies username "bob" via RSS generator, WP JSON API, author ID brute forcing, and login error message analysis.

5. Credential Brute-Force Attack

5.1 WPScan XML-RPC Password Attack

With username "bob" confirmed, WPScan was re-run in password brute-force mode, targeting bob's account using the rockyou.txt wordlist. WPScan attacked the WordPress XML-RPC endpoint (xmlrpc.php) rather than the login page, which is significantly faster and bypasses many simple rate-limiting controls.

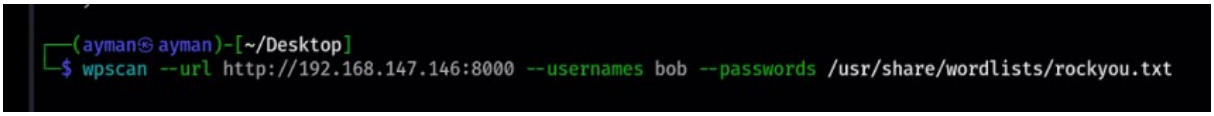

Figure 8 — WPScan brute-force command: targeting user "bob" with the rockyou.txt wordlist against http://192.168.147.146:8000.

The attack succeeded: WPScan found the valid credential pair bob / Welcome1. No account lockout or CAPTCHA was present on the XML-RPC endpoint.

Figure 9 — WPScan brute-force result: [SUCCESS] username "bob", password "Welcome1" recovered via XML-RPC attack after approximately 40,400 attempts.

5.2 WordPress Admin Panel Access

The compromised credentials were used to log into the WordPress admin dashboard at http://192.168.147.146:8000/wp-admin/. Full administrative access was confirmed — "Howdy, bob" displayed in the top-right corner.

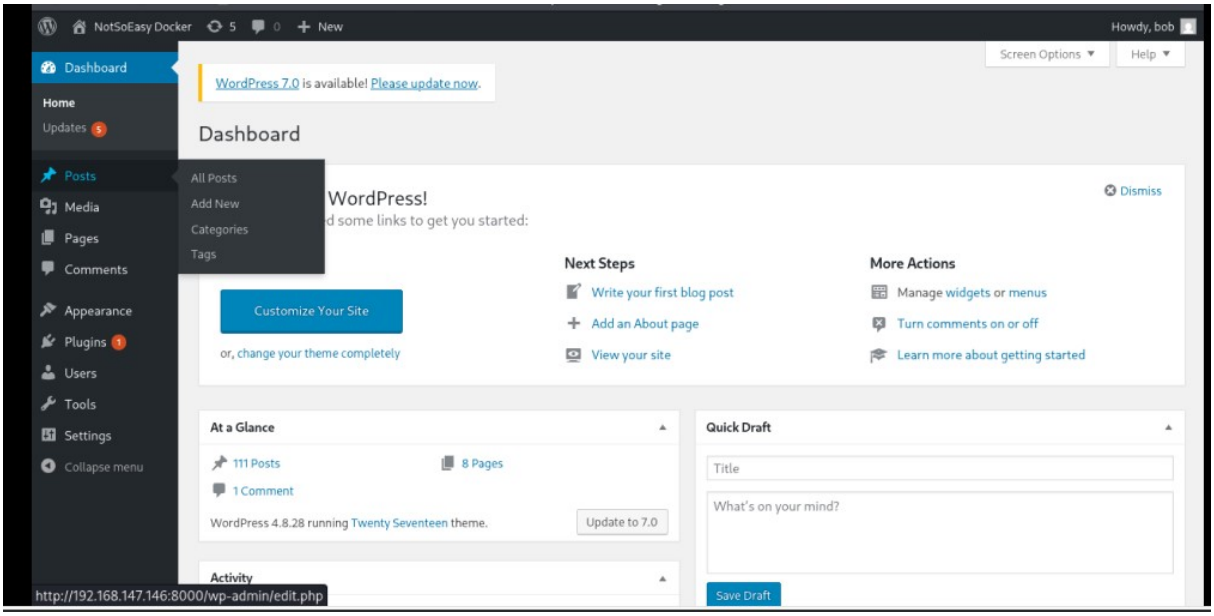

Figure 10 — WordPress admin dashboard accessed as user "bob." Site: NotSoEasy Docker, WordPress 4.8.1, 111 posts, 8 pages visible.

6. Exploitation — Remote Code Execution

6.1 Metasploit wp_admin_shell_upload

The Metasploit module exploit/unix/webapp/wp_admin_shell_upload was used to gain a reverse Meterpreter shell. This module authenticates to WordPress with the compromised credentials, uploads a malicious PHP plugin, and triggers it to call back to the attacker. The LHOST was initially set incorrectly (172.16.0.2 — a VMware NAT address not reachable by the target); after correction to 192.168.147.129 (the Kali bridged adapter address), the exploit succeeded.

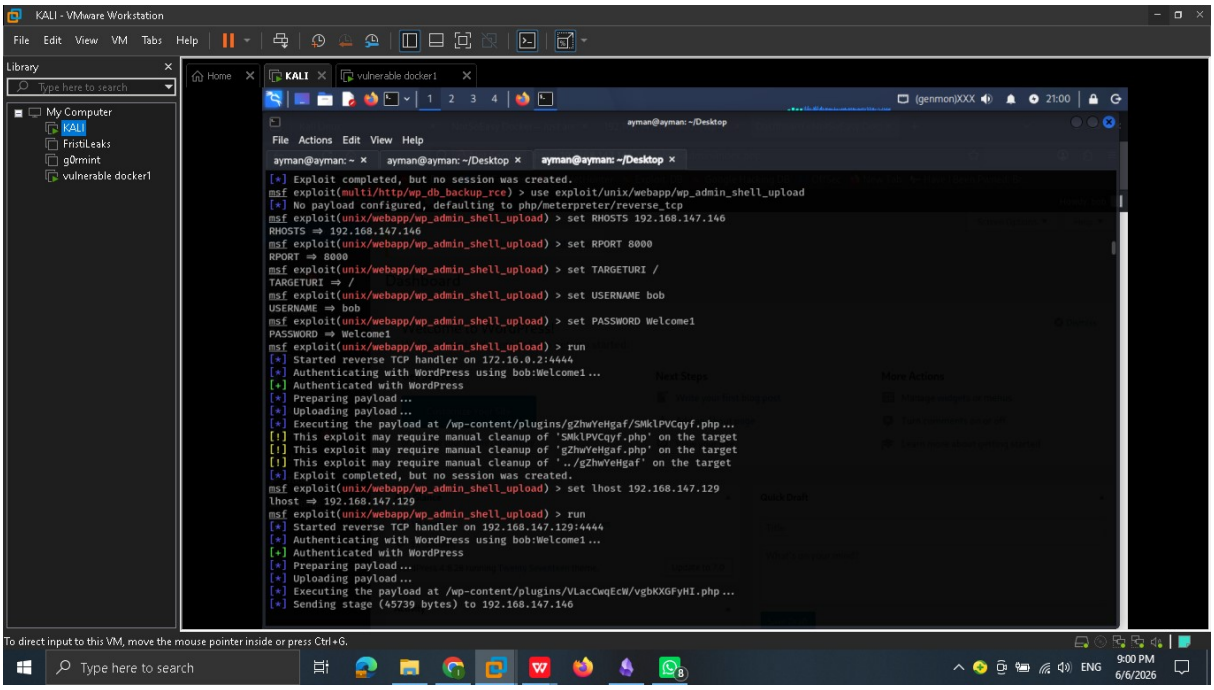

Figure 11 — Metasploit wp_admin_shell_upload: RHOSTS=192.168.147.146, RPORT=8000, USERNAME=bob, PASSWORD=Welcome1. Initial LHOST misconfiguration corrected to 192.168.147.129. Payload uploaded and stage sent successfully.

6.2 Meterpreter Shell — Initial Access

A Meterpreter session was established as user www-data (the Apache web process user) on host 8f4bca8ef241. The sysinfo command confirmed Linux kernel 3.13.0-128-generic (Ubuntu), x86_64 architecture. The getuid command confirmed the running identity as www-data.

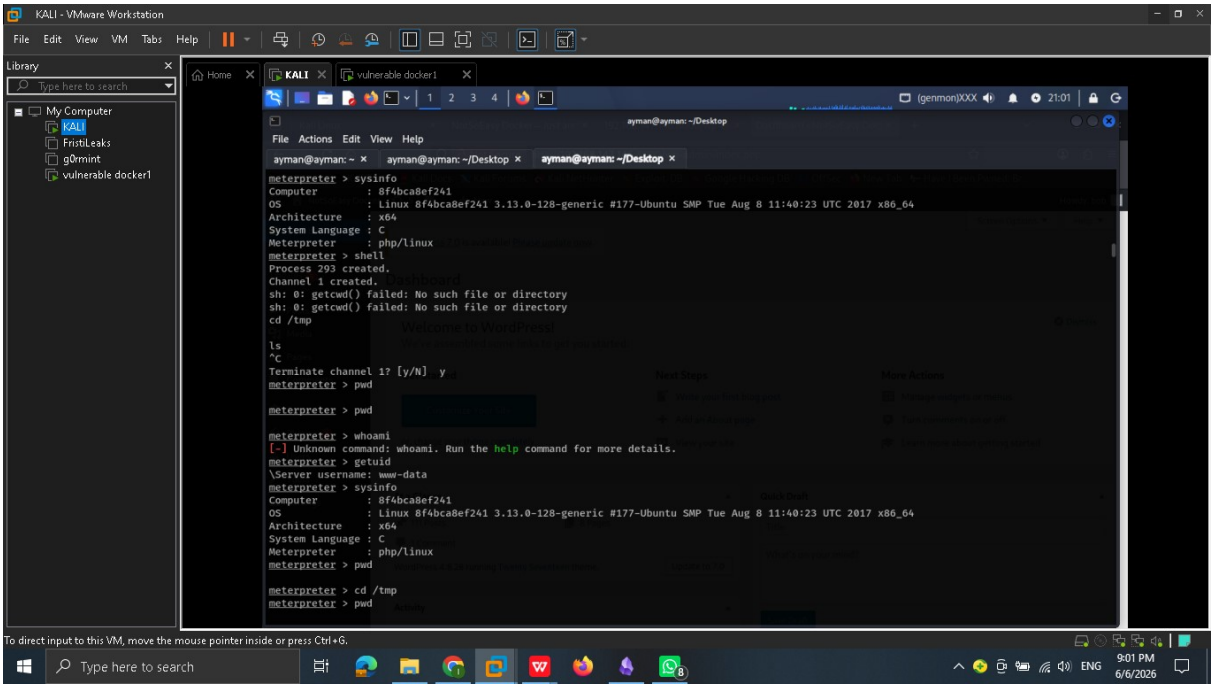

Figure 12 — Meterpreter session established. sysinfo: Linux 3.13.0-128-generic, x86_64. getuid: Server username www-data. Shell spawned, navigated to /tmp.

7. Post-Exploitation & Internal Discovery

7.1 System Survey & OS Identification

Within the Meterpreter shell, a bash shell was spawned and basic enumeration was performed. Python was not available on the target. The OS was identified via uname -a and /etc/os-release as Debian GNU/Linux 8 (Jessie) — confirming this is a Docker container (not the host OS). The container hostname 8f4bca8ef241 is a Docker container ID.

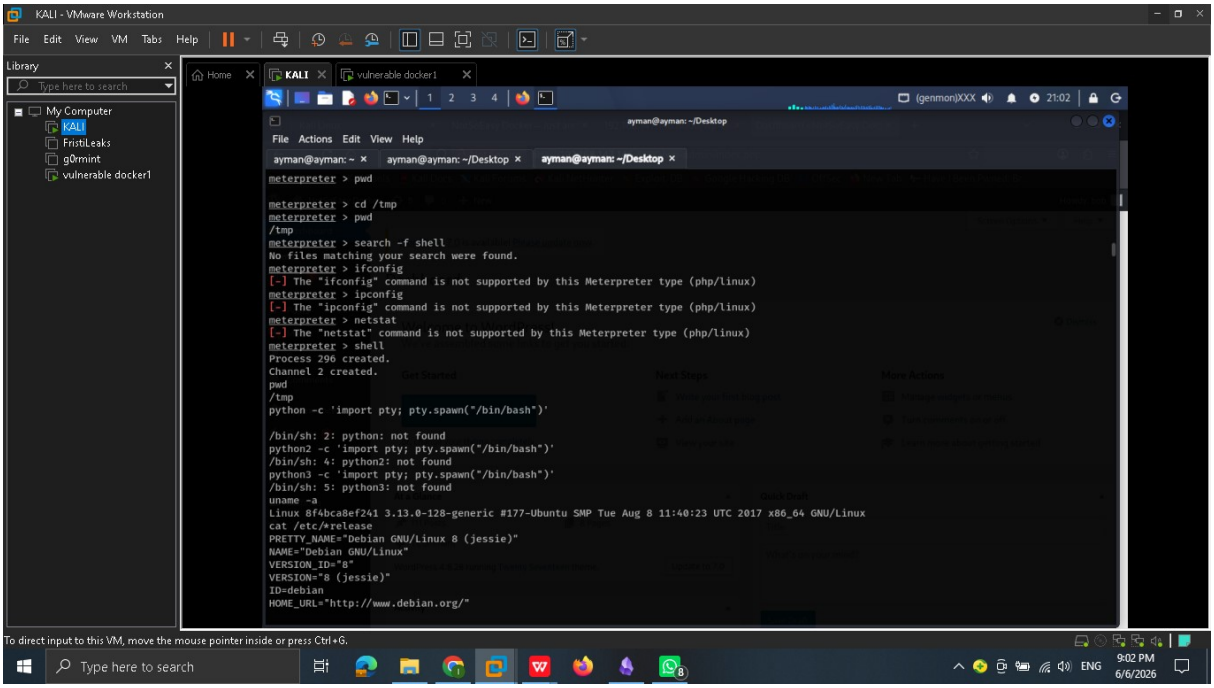

Figure 13 — Shell inside container: uname reveals Linux 3.13.0-128-generic; /etc/os-release shows Debian GNU/Linux 8 (Jessie). Python unavailable. Container ID: 8f4bca8ef241.

7.2 SUID & Network Enumeration

A find command was issued to enumerate SUID binaries (find / -perm -4000 -type f 2>/dev/null), returning standard Linux utilities: chsh, passwd, gpasswd, newgrp, chfn, umount, mount, ping, ping6, su. Additionally, ss -tuln was used to view listening services inside the container — revealing MySQL on 127.0.0.1:3306 and a service on port 80.

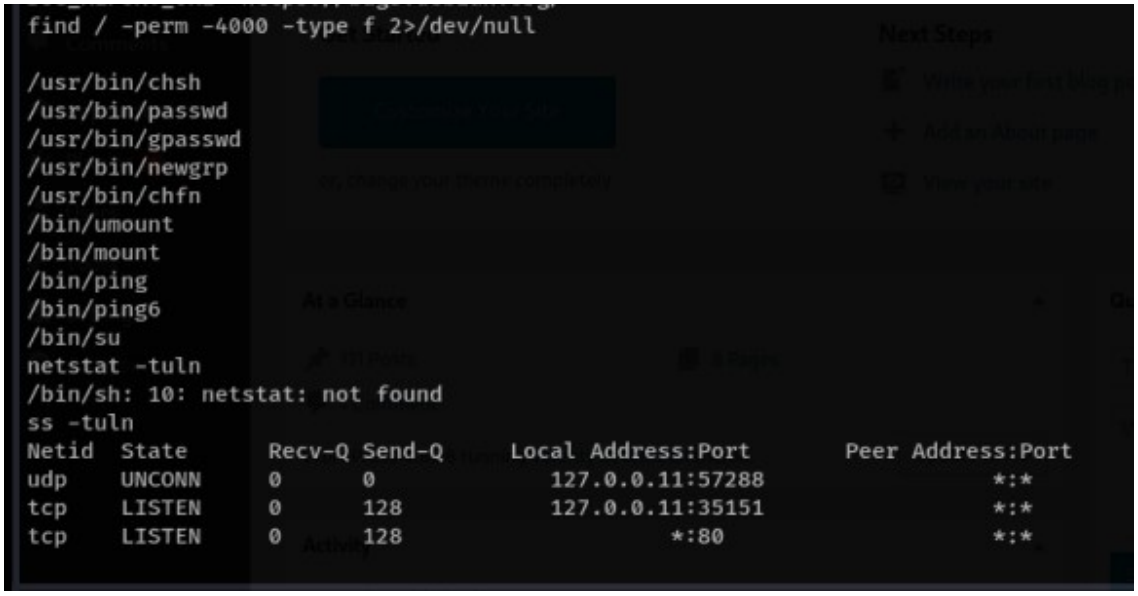

Figure 14 — SUID enumeration and ss -tuln output: TCP ports 35151 and 80 listening inside the container; no unusual SUID binaries found.

7.3 LinEnum Script Upload

To accelerate internal enumeration, LinEnum.sh was downloaded from GitHub raw content using curl into the Kali session, renamed, made executable, and prepared for transfer to the target container.

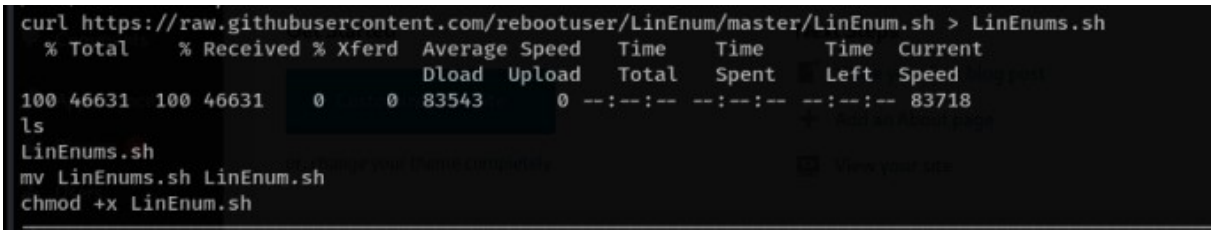

Figure 15 — LinEnum.sh downloaded from GitHub raw URL using curl, renamed to LinEnum.sh, and chmod +x applied.

7.4 Internal Network Interface Discovery

The ip addr command run inside the container's shell revealed two network interfaces: the loopback (lo: 127.0.0.1/8) and eth0 (172.18.0.2/16). The 172.18.0.0/16 subnet is the internal Docker bridge network — invisible from the external 192.168.147.0/24 interface. This is a classic Docker multi-container topology where the host forwards only selected ports from the external interface.

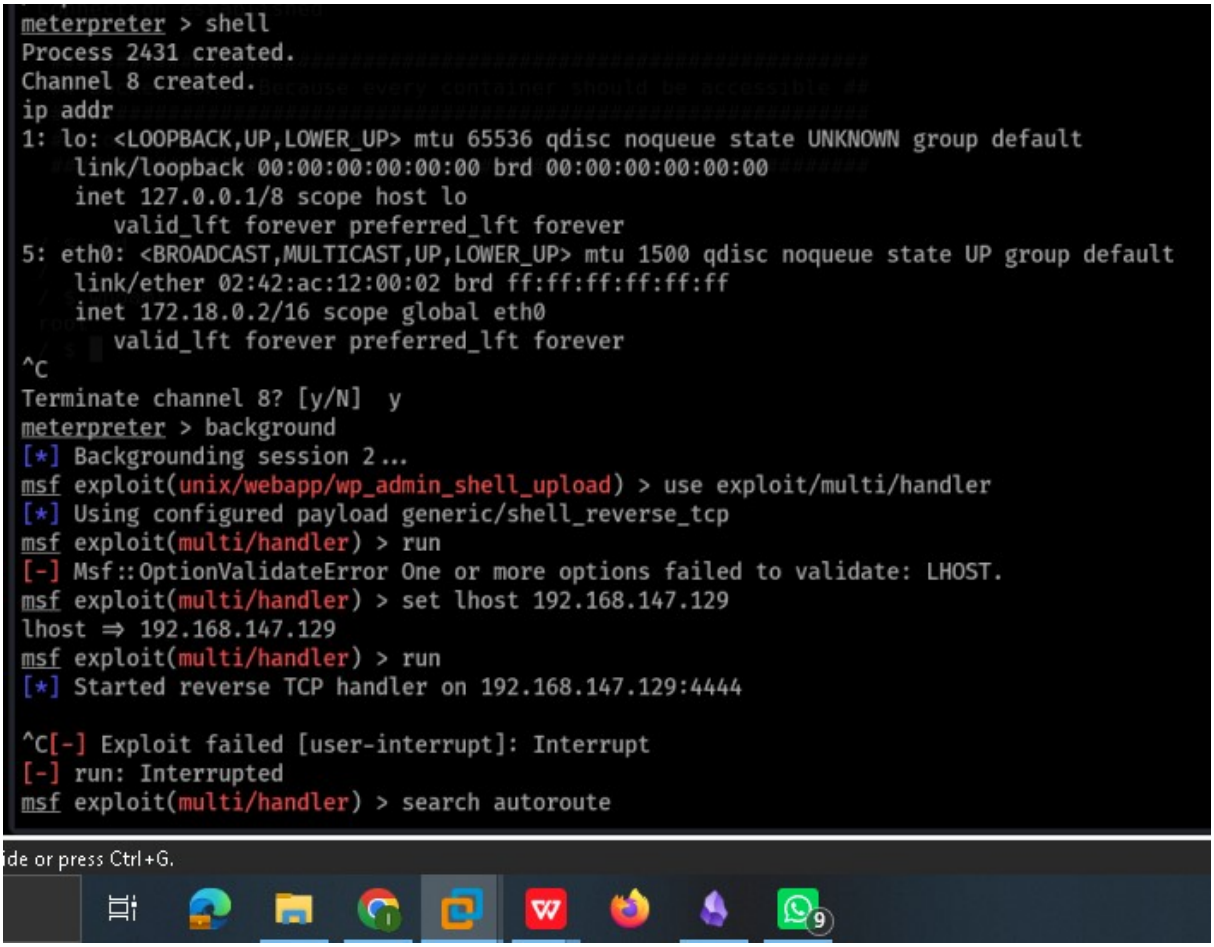

Figure 16 — ip addr inside container: eth0 is 172.18.0.2/16 (Docker internal bridge). Meterpreter session backgrounded for pivoting.

8. Network Pivoting

8.1 Metasploit Autoroute Setup

The Metasploit post/multi/manage/autoroute module was used to add a routing entry for the internal 172.18.0.0/255.255.0.0 subnet through the existing Meterpreter session. This allows Metasploit to tunnel subsequent scans and connections through the compromised container.

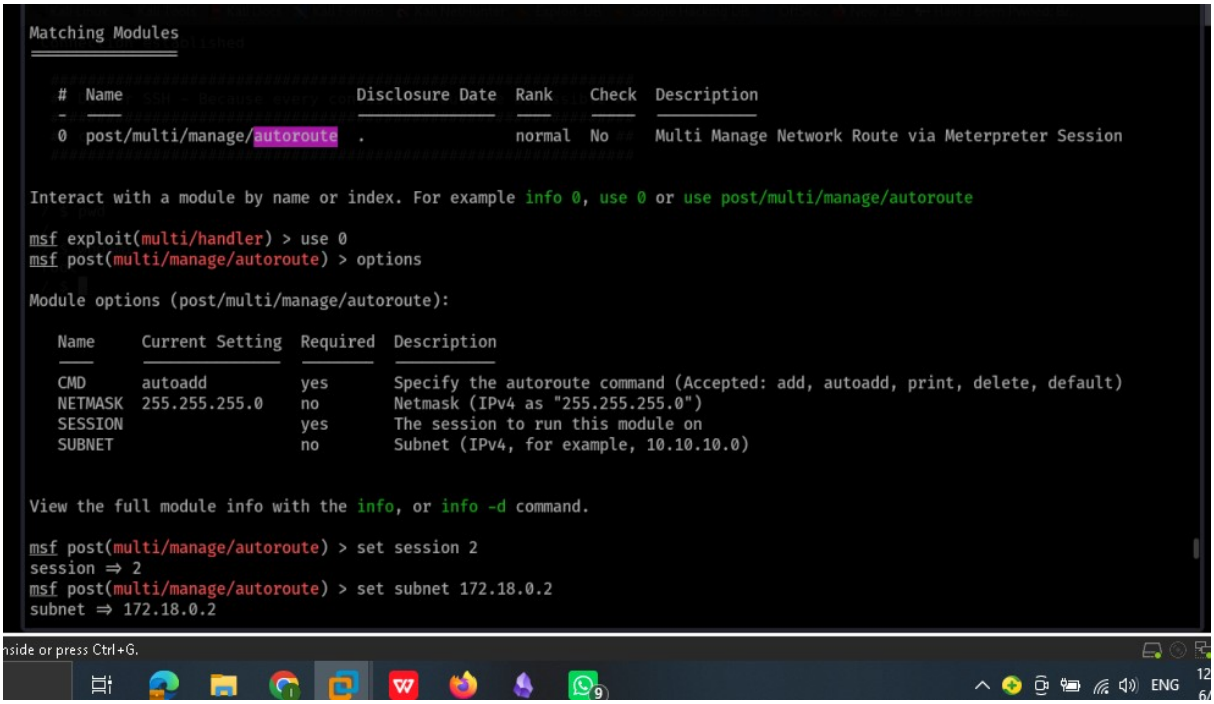

Figure 17 — autoroute module configured: SESSION=2, SUBNET=172.18.0.2 (Docker internal bridge). Route will be injected for 172.18.0.0/16.

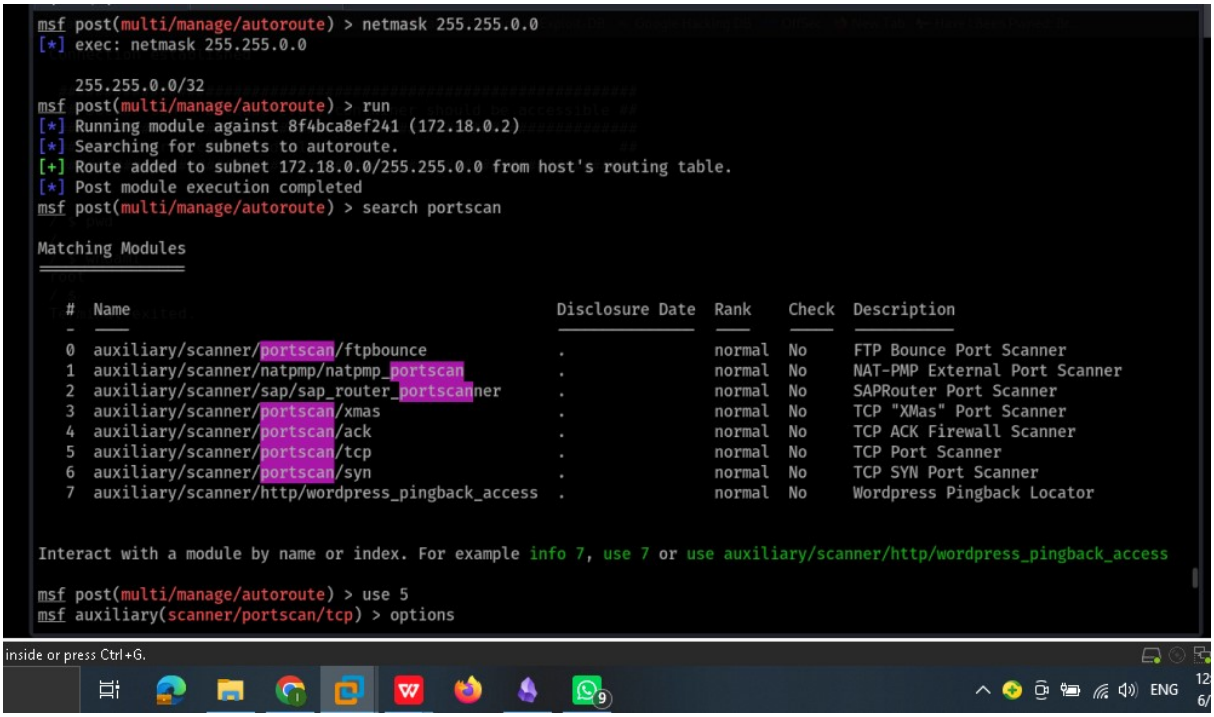

Figure 18 — autoroute executed: route added for 172.18.0.0/255.255.0.0 via session 2. Metasploit TCP port scanner (portscan/tcp) selected for internal subnet scan.

8.2 Internal Subnet Port Scan

The auxiliary/scanner/portscan/tcp module was configured to scan 172.18.0.2/24 for common ports. The scan discovered four live internal hosts: 172.18.0.1 (SSH:22, HTTP:8000), 172.18.0.2 (the compromised container, HTTP:80), 172.18.0.3 (MySQL:3306), and 172.18.0.4 (SSH:22, port 8022).

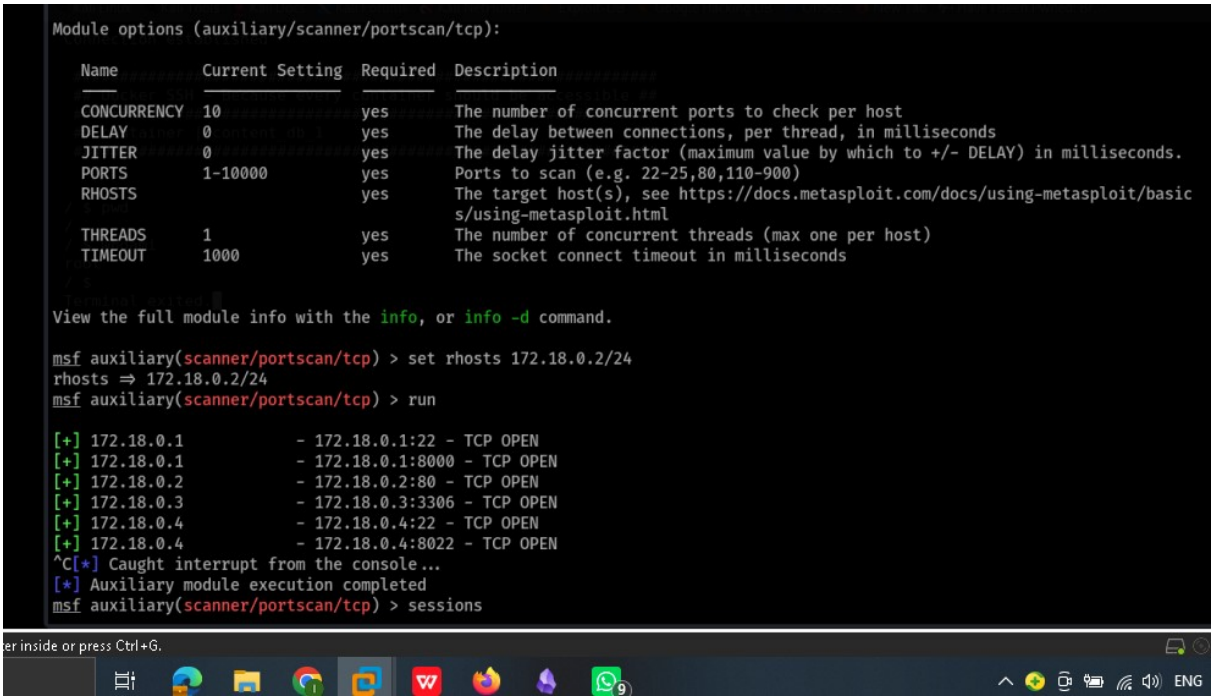

Figure 19 — Internal TCP scan results: 172.18.0.1 (ports 22, 8000), 172.18.0.2 (port 80), 172.18.0.3 (port 3306 MySQL), 172.18.0.4 (ports 22, 8022). Three active sessions established.

8.3 Port Forwarding to Target Container

With three active Meterpreter sessions confirmed, session 2 was selected and the portfwd feature was used to forward local port 8022 on the Kali machine to 172.18.0.4:8022 (the internal Docker container). This makes the internal container's SSH service accessible directly from the attacker's machine as if it were local.

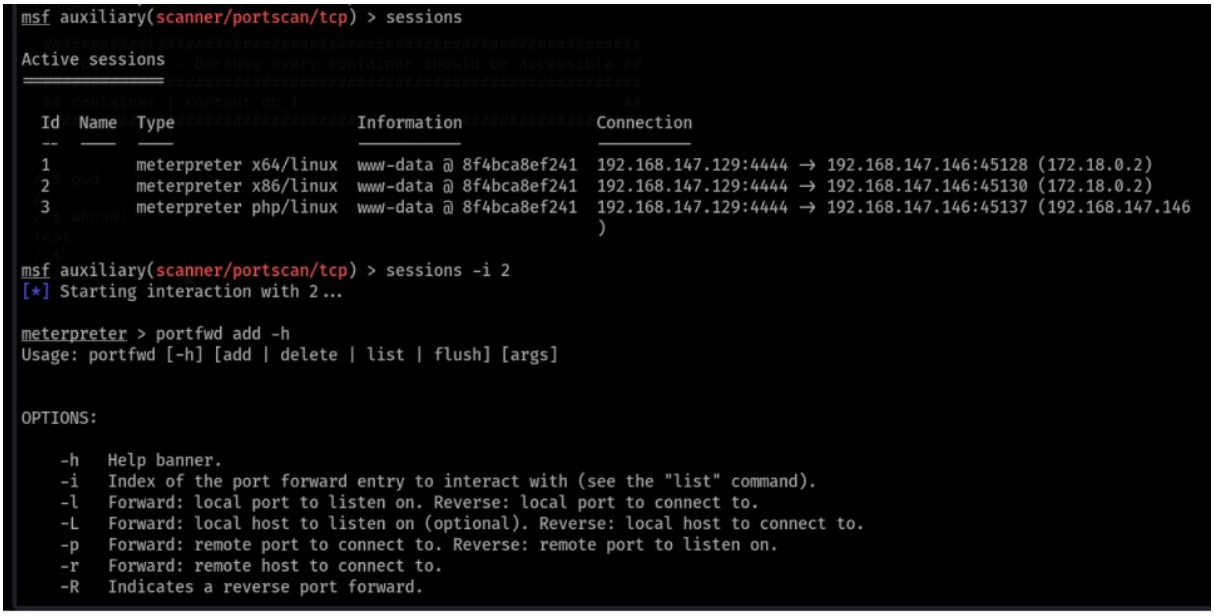

Figure 20 — Active sessions: three Meterpreter sessions as www-data. Session 2 selected; portfwd -h options reviewed.

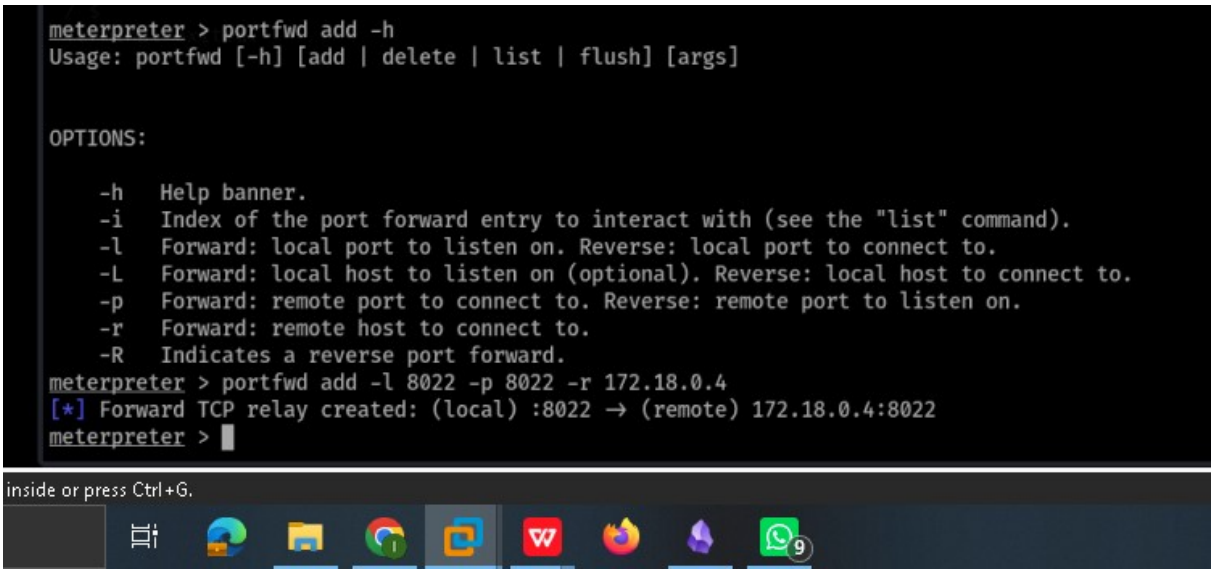

Figure 21 — portfwd add -l 8022 -p 8022 -r 172.18.0.4: Forward TCP relay created — local :8022 → remote 172.18.0.4:8022.

9. Root Access — Docker Container Escape

9.1 SSH Login to Internal Container as Root

With port forwarding active, the attacker navigated to http://192.168.147.129:8022 in the browser. The service presented a "Docker SSH" banner — "Because every container should be accessible" — and the label "container | content db 1." No password was required. A shell was obtained as root (uid=0, gid=0). Running whoami and id confirmed full root access inside the internal database container.

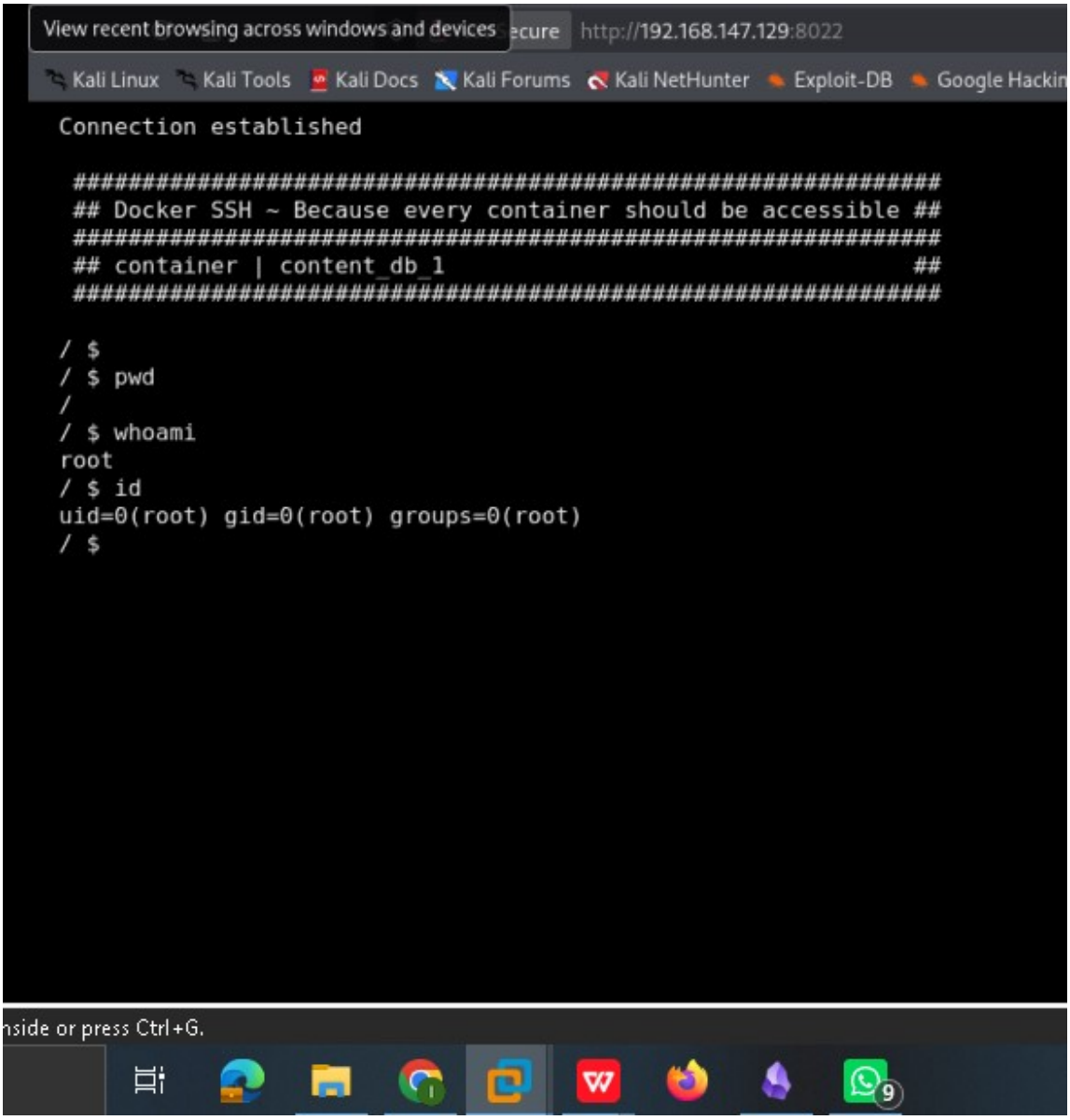

Figure 22 — SSH service on 172.18.0.4:8022 (forwarded to localhost:8022) responds with Docker SSH banner. whoami = root, id = uid=0(root) gid=0(root) groups=0(root). Root access achieved inside container "content db 1."

10. Findings Summary

#	Finding	Severity	Impact
1	WordPress User Enumeration via XML-RPC	High	Enabled attacker to identify valid username "bob" for targeted brute-force.
2	Weak WordPress Credentials (bob / Welcome1)	Critical	Brute-forced in ~40,000 attempts; granted full WordPress admin access.
3	Outdated WordPress (4.8.1) + Apache (2.4.10)	High	Vulnerable to known exploits; wp_admin_shell_upload module fully functional.
4	Unauthenticated File Upload via WordPress Admin	Critical	Enabled remote code execution as www-data inside the web container.
5	Docker Internal Network Exposure (172.18.0.0/16)	Critical	Internal containers reachable after initial compromise; lateral movement possible.
6	Docker SSH Container with No Authentication	Critical	172.18.0.4:8022 allowed root login with no password — trivial full container compromise.

11. Recommendations

11.1 Immediate (Critical)

- Change all WordPress credentials immediately. Enforce a minimum 12-character password policy with complexity requirements.
- Disable or restrict WordPress XML-RPC (xmlrpc.php) at the web server level if it is not required for legitimate use.
- Remove or reconfigure the Docker SSH container (172.18.0.4:8022). No container should expose an unauthenticated root SSH shell on any network.
- Implement Docker network segmentation: use custom bridge networks with explicit allow-lists rather than exposing all containers to a flat internal LAN.

11.2 Short-Term (High)

- Update WordPress to the latest stable release and apply all plugin/theme updates.
- Update Apache httpd and PHP to current supported versions (Apache 2.4.10 and PHP 5.6.31 are both end-of-life).
- Enable a Web Application Firewall (WAF) rule to rate-limit and alert on repeated failed login attempts at the WordPress login page and xmlrpc.php.
- Restrict Meterpreter-reachable pivoting by applying strict egress firewall rules from each container — containers should only be able to initiate outbound connections to known, required services.

11.3 Long-Term (Defense-in-Depth)

- Adopt a secrets management solution (e.g., HashiCorp Vault) — avoid hardcoding or reusing credentials across WordPress, database, and SSH services.
- Implement container image scanning and runtime security (e.g., Trivy, Falco) to detect and alert on abnormal process execution (e.g., reverse shells, new listening ports).
- Conduct periodic penetration tests and purple-team exercises to validate that container escape paths remain closed as the environment evolves.

12. Conclusion

This assessment of the VulnHub Vulnerable Docker 1 machine demonstrated that a single weak WordPress credential was sufficient to escalate from zero access to root inside a hidden Docker container on an internal network. No kernel exploits were required. The attack relied entirely on misconfiguration, weak credentials, and the absence of network egress controls between containers.

The findings emphasize that containerization is not a security boundary by itself. Each container must be hardened independently, network access between containers must be minimized, and credentials must meet modern strength and rotation standards. Addressing the recommendations in Section 11 will significantly reduce the attack surface of this environment.

— End of Report —